



Работа со срезами

Время изучения: 16 мин

Автор: Матвей Ефимов, Разработчик учебных программ и эксперт в Go, Python, SQL

Вы изучите:

- [Определение среза](#)
- [Создание срезов](#)
- [Основные операции со срезами](#)
- [Практические примеры](#)
- [Важные нюансы](#)



Определение среза

В языке Go **срез (slice)** — это динамическая структура данных, представляющая собой **последовательность элементов одного типа**, как и массив. Но, в отличие от массивов, срезы гибкие по размеру — вы можете добавлять и удалять элементы, не создавая новый массив.

Можно сказать, что **срез — это «обёртка» над массивом**, которая содержит:

- указатель на элементы массива;
- длину (количество элементов);
- ёмкость (максимальное количество элементов, которые можно разместить без перераспределения памяти).

Срезы — один из самых часто используемых инструментов в Go, потому что они удобнее и безопаснее, чем массивы в большинстве практических задач.

Отличия между массивами и срезами:

Характеристика	Массив ([N]T)	Срез ([]T)
Размер	Фиксированный	Динамический (может меняться)
Передача в функции	Копируется полностью	Передаётся по ссылке
Гибкость	Неудобны для изменения длины	Можно легко добавлять/удалять элементы
Использование в коде	Реже используется напрямую	Основной способ хранения последовательностей
Память	Всегда содержит все элементы	Может ссылаться на часть массива

Когда использовать массив, а когда — срез:

Использование	Рекомендуется
Фиксированное количество элементов, например, [3]float64 для векторов	Массив
Размер заранее неизвестен или может меняться	Срез
Нужно передавать данные в функции без копирования	Срез
Работа с подмножеством массива или другим срезом	Срез

В большинстве случаев массивы в языке Go менее удобны, чем срезы.
Причины:

- При передаче массива в функцию он **копируется полностью**, что неэффективно.
- Размер массива нужно указывать явно, и он не может меняться.
- Большая часть стандартных функций в Go работает со срезами, а не с массивами.

Поэтому **на практике почти всегда используют срезы**, а массивы — только если нужно работать с фиксированным числом элементов, например, для оптимизации производительности или в системном программировании.

Создание срезов

Срезы можно создавать разными способами. В этом разделе разберём три основных варианта:

1. Создание среза **из массива**.
2. Создание среза **через литерал**.
3. Создание среза **с помощью функции make()**.

1. Создание среза из массива

```
package main

import "fmt"

func main() {
    arr := [5]int{10, 20, 30, 40, 50} // создаём массив из 5
элементов
    slice := arr[1:4] // создаём срез с
элементами от индекса 1 до 3 (не включая 4)

    fmt.Println(slice) // [20 30 40]
}
```

Пояснения:

- `arr := [5]int{...}` — создаём массив из 5 целых чисел.
- `arr[1:4]` — берём элементы с индексами **1, 2, 3**. Элемент с индексом 4 не входит.
- `slice` — это срез, указывающий на часть массива `arr`.
- `fmt.Println(slice)` — печатает срез: `[20 30 40]`.

Важно: Срез не копирует данные — он ссылается на часть массива. Это значит, что изменения через срез повлияют на массив. Мы подробнее разберём это позже.

2. Создание среза через литерал

Срез можно создать сразу с нужными значениями, не указывая размер.

```
package main

import "fmt"

func main() {
    slice := []string{"яблоко", "банан", "вишня"} // создаём срез
строк
    fmt.Println(slice) // [яблоко банан вишня]
}
```

Пояснения:

- `[]string{...}` — срез строк. Квадратные скобки **пустые** — это и есть признак среза;
- указаны три элемента: яблоко, банан, вишня;
- `slice` — это срез длиной 3;

- `fmt.Println(slice)` — выведет: [яблоко банан вишня].

Этот способ самый удобный, если сразу известны значения.

3. Создание среза с помощью `make()`

Функция `make()` используется, чтобы создать срез с определённой длиной и ёмкостью.

```
package main

import "fmt"

func main() {
    slice := make([]int, 3, 5) // создаём срез из 3 элементов,
    ёмкостью 5
    fmt.Println(slice)         // [0 0 0]
}
```

Пояснения:

- `make([]int, 3, 5)` — создаёт срез из **int**, длиной **3**, ёмкостью **5**;
- длина — это сколько элементов доступно сразу;
- ёмкость — сколько элементов можно добавить до перераспределения памяти;
- все элементы по умолчанию инициализируются нулями (для `int` это 0);
- `fmt.Println(slice)` — выведет: [0 0 0].

Если указать только два аргумента (длину и тип), ёмкость будет равна длине:

```
s := make([]int, 4) // длина и ёмкость равны 4
```

Теперь вы знаете три способа создать срез:

Способ	Пример	Когда использовать
Из массива	<code>arr[1:4]</code>	Когда уже есть массив, и нужна его часть
Через литерал	<code>[]int{1, 2, 3}</code>	Когда вы заранее знаете все значения
Через <code>make()</code>	<code>make([]int, 3, 5)</code>	Когда нужно создать пустой срез определённой длины и ёмкости

Основные операции со срезами

В этом разделе рассмотрим **ключевые действия**, которые часто выполняются при работе со срезами:

1. получение длины и ёмкости (len, cap);
2. добавление элементов (append);
3. срезание (slicing);
4. копирование срезов (copy);
5. итерация по срезу (цикл for).

1. Длина (len) и ёмкость (cap)

```
package main

import "fmt"

func main() {
    slice := make([]int, 3, 5) // создаём срез длиной 3 и
                               // ёмкостью 5

    fmt.Println("Длина:", len(slice)) // 3
    fmt.Println("Ёмкость:", cap(slice)) // 5
}
```

Пояснение:

- len(slice) — возвращает количество доступных элементов;
- cap(slice) — показывает, сколько элементов может вмещать срез без выделения новой памяти;
- в этом примере: длина 3, ёмкость 5.

2. Добавление элементов (append)

```
package main

import "fmt"

func main() {
    slice := []int{1, 2, 3} // создаём срез из трёх чисел
    slice = append(slice, 4, 5) // добавляем два элемента
    fmt.Println(slice)          // [1 2 3 4 5]
}
```

Пояснение:

- `append(slice, 4, 5)` — добавляет 4 и 5 в конец среза;
- результат нужно сохранить обратно, так как может быть создан новый срез;
- `append` работает с переменным числом аргументов.

Что происходит при переполнении ёмкости?

```
package main

import "fmt"

func main() {
    slice := make([]int, 2, 2) // длина и ёмкость 2
    fmt.Println("До append:", slice, "cap:", cap(slice))

    slice = append(slice, 100) // добавляем третий элемент →
    // ёмкость будет увеличена
    fmt.Println("После append:", slice, "cap:", cap(slice))
}
```

Пояснения:

- При добавлении элементов сверх ёмкости, Go автоматически выделяет новый массив, часто с удвоенной ёмкостью.
- Поэтому `cap(slice)` после `append` может измениться (например, с 2 до 4).

3. Срезание (slicing)

```
package main

import "fmt"

func main() {
    nums := []int{10, 20, 30, 40, 50}
    part := nums[1:4] // берём элементы с индексами 1, 2, 3
    fmt.Println(part) // [20 30 40]
}
```

Пояснения:

- `nums[1:4]` — создаёт новый срез от индекса 1 включительно до 4 не включительно;
- получаем 20, 30, 40;
- в Go принято обозначать slicing как `slice[start:end]`, где `start` включается, а `end` — не включается.

4. Копирование срезов (copy)

```
package main

import "fmt"

func main() {
    src := []int{1, 2, 3}
    dst := make([]int, 3) // создаём срез-назначение нужной длины

    copied := copy(dst, src) // копируем src → dst
    fmt.Println("dst:", dst) // [1 2 3]
    fmt.Println("Скопировано элементов:", copied) // 3
}
```

Пояснения:

- `copy(dst, src)` — копирует из `src` в `dst`;
- количество реально скопированных элементов — это `min(len(src), len(dst))`;
- результат — число скопированных элементов.

5. Итерация по срезу

```
package main

import "fmt"

func main() {
    slice := []string{"a", "b", "c"}

    for i, value := range slice {
        fmt.Println("Индекс:", i, "Значение:", value)
    }
}
```

Пояснения:

- `range` позволяет пройти по всем элементам среза;
- `i` — индекс (0, 1, 2...);
- `value` — значение по текущему индексу.

Если индекс не нужен:

```
for _, value := range slice {
    fmt.Println(value)
}
```

Практические примеры

После того как мы изучили основные операции со срезами, давайте посмотрим, как они применяются на практике.

Пример 1: Создание и модификация среза

```
package main

import "fmt"

func main() {
    numbers := []int{1, 2, 3} // создаём срез из трёх чисел

    numbers[0] = 10           // изменяем первый элемент
    numbers[2] = 30           // изменяем третий элемент

    fmt.Println(numbers)      // [10 2 30]
}
```

Пояснения:

- срез numbers можно изменять напрямую по индексу;
- мы изменили элементы с индексами 0 и 2;
- срез — это не константа, его содержимое можно менять.

Пример 2: Добавление элементов с помощью append и копирование сору

```
package main

import "fmt"

func main() {
    a := []int{1, 2, 3}           // создаём срез
    b := append(a, 4, 5)          // добавляем два элемента
    fmt.Println("b:", b)          // [1 2 3 4 5]

    c := make([]int, len(b))      // создаём новый срез нужной
    // длины
    copy(c, b)                    // копируем элементы из b в c
    fmt.Println("c:", c)          // [1 2 3 4 5]
}
```

Пояснения:

- append(a, 4, 5) — создаёт новый срез b, расширяя a;
- make([]int, len(b)) — создаём срез нужной длины для копирования;
- copy(c, b) — копируем все элементы из b в c.

Пример 3: Фильтрация чётных чисел

Рассмотрим простой пример, где из среза удаляются все нечётные числа и остаются только чётные.

```
package main

import "fmt"

func main() {
    numbers := []int{1, 2, 3, 4, 5, 6} // исходный срез
    evens := []int{}                  // создаём пустой срез для
    чётных чисел

    for _, num := range numbers { // обходим все элементы
        // исходного среза
        if num%2 == 0 {           // проверяем, делится ли число на 2
            // без остатка
            evens = append(evens, num) // добавляем чётное число
            // в новый срез
        }
    }

    fmt.Println("Чётные числа:", evens) // [2 4 6]
}
```

Объяснение:

- мы создаём новый срез `evens`, в который будем добавлять только нужные элементы;
- `if num%2 == 0` — условие для проверки чётности;
- `append(evens, num)` — добавляем элемент, если он подходит;
- такой подход часто используется для фильтрации данных.

Важные нюансы

Срезы в Go — мощный инструмент, но за простым синтаксисом скрываются **особенности**, которые важно понимать. Ниже — ключевые моменты, которые помогут избежать ошибок.

1. Изменение среза влияет на исходный массив (если срез создан из массива)

```
package main

import "fmt"

func main() {
    arr := [5]int{1, 2, 3, 4, 5} // обычный массив
    slice := arr[1:4]           // создаём срез с элементами 2, 3, 4

    slice[0] = 100              // меняем первый элемент среза
    // (это arr[1])

    fmt.Println("Массив:", arr) // [1 100 3 4 5]
    fmt.Println("Срез:", slice)  // [100 3 4]
}
```

Почему так происходит:

- срез не копирует данные — он указывает на ту же область памяти;
- изменение среза = изменение исходного массива.

2. append может вызвать переаллокацию памяти

```
package main

import "fmt"

func main() {
    base := []int{1, 2, 3}
    sub := base[:2] // создаём срез из первых двух элементов

    sub = append(sub, 100, 200) // добавляем два элемента —
    // может произойти переаллокация

    fmt.Println("base:", base) // [1 2 3]
    fmt.Println("sub:", sub)    // [1 2 100 200]
}
```

Что происходит:

- Если в sub не хватает ёмкости для append, Go выделяет новую память, и sub становится независимым от base.

- В этом примере base остаётся неизменным, потому что append вызвал переаллокацию.

3. Пустой срез ([]T{}) vs nil-срез (var s []T)

```
package main

import "fmt"

func main() {
    var a []int           // nil-срез
    b := []int{}          // пустой срез, не nil

    fmt.Println("a == nil:", a == nil) // true
    fmt.Println("b == nil:", b == nil) // false

    fmt.Println("len(a):", len(a))      // 0
    fmt.Println("len(b):", len(b))      // 0
}
```

В чём разница:

- `var a []int` — это nil-срез. Он не указывает ни на какой массив.
- `b := []int{}` — пустой срез, но он указывает на выделенную (пустую) память.
- Оба имеют длину 0, но nil-срез можно использовать, например, чтобы обозначить «ничего нет».

4. Срез может использовать только часть ёмкости массива

```
package main

import "fmt"

func main() {
    arr := [5]int{1, 2, 3, 4, 5}
    s := arr[1:3]           // элементы 2, 3

    fmt.Println("len:", len(s)) // 2
    fmt.Println("cap:", cap(s)) // 4 (считает от s[0] до конца массива)
}
```

Важно помнить:

- Длина `len(s)` — количество видимых элементов ($3-1 = 2$).
- Ёмкость `cap(s)` — сколько всего можно добавить, начиная от `s[0]` до конца исходного массива.

- Это важно при append: если не выйти за пределы cap, изменения будут затрагивать исходный массив.

5. Функции работают с копией среза, но с теми же данными

```
package main

import "fmt"

func modify(s []int) {
    s[0] = 999          // изменяем первый элемент
}

func main() {
    nums := []int{1, 2, 3}
    modify(nums)        // передаём срез в функцию

    fmt.Println(nums)   // [999 2 3]
}
```

Что происходит:

- В функцию передаётся копия среза, но она указывает на ту же область памяти.
- Поэтому изменения внутри функции влияют на исходный срез.

Вывод:

Особенность	Объяснение
Срез ссылается на массив	Изменения среза затрагивают массив
append может переаллоцировать	Тогда создаётся новый массив, и срез становится независимым
nil vs пустой срез	Оба длины 0, но nil — отсутствие данных, а пустой срез — данные есть
cap может быть больше len	Можно добавлять элементы, если есть запас ёмкости
Передача в функцию	Передаётся копия структуры, но данные общие

Подведём итоги

- Срез (slice) — это гибкая, динамически расширяемая структура, которая ссылается на часть массива.
- Срезы удобнее массивов, когда нужно работать с изменяемыми последовательностями данных.
- Основные операции: создание срезов, получение длины и ёмкости, добавление элементов через `append`, копирование с помощью `copy`.
- Изменения в срезе влияют на исходный массив, если срез создан из массива и не произошло переаллокации.
- `append` может создавать новый массив, если не хватает места — это важный момент для понимания работы с памятью.
- Есть различие между пустым срезом и `nil`-срезом, что важно для проверки и логики в коде.
- Срезы часто передаются в функции, где копируется структура среза, но не данные — изменения в данных видны и снаружи.

Полезные ссылки:

- [Официальная документация Go \(на английском\)](#)
- [Раздел о срезах в Go Tour \(с интерактивными примерами\)](#)

Мини-задача для практики:

Напишите программу, которая:

- Создаёт срез из чисел от 1 до 10.
- Использует `append` для добавления чисел 11 и 12.
- Создаёт новый срез, состоящий из чисел с индексами 3 по 7 (включительно).
- Копирует этот новый срез в другой срез.
- Изменяет первый элемент копии.
- Выводит все срезы и показывает, что произошло с исходными данными.